CSE 4303
Data Structures

Topic: Range Queries

(Prefix sum, Sparse Table)

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

Prefix Sum

- Efficiently computes ***sum of elements in a subarray***.

- Used in problems related to ***range sum queries***, cumulative frequency calculations, etc.

- Answers *sum queries* in constant time $O(1)$, after an initial preprocessing step of $O(N)$.

pref[k] = sum of first k items

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	?	?	?	?	?	?	?	?

pref[0] = 0

pref[i] = ? // general formula

pref[k] = sum of first k items

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

```
pref[0] = 0
pref[i] = ?      // general formula
```

pref[k] = sum of first k items

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

pref[0] = 0

pref[i] = pref[i-1] + a[i-1]

```
Find sum(L,R)          // let (L,R)=(2,5)
```

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Find sum(L,R) // let (L,R)=(2,5)

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

```
Sum(2,5)   = sum of items upto idx 5 –
             sum of items upto idx 1
           = 14 – 5 = 9  // (1+3+1+4=9)
```

Find sum(L,R) // let (L,R)=(2,5)

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Sum(2,5) = sum of items upto idx 5 –
sum of items upto idx 1

Sum(L,R) = ? // in general?

Find sum(L,R) // let (L,R)=(2,5)

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Sum(2,5) = sum of items upto idx 5 – *// stored in index 6, i.e., index $(R+1)$*

sum of items upto idx 1 *// stored in index 2, i.e., index (L)*

Sum(L,R) = ? // in general?

Find sum(L,R) // let (L,R)=(2,5)

idx	0	1	2	3	4	5	6	7
a[i]	3	2	1	3	1	4	2	
pref[i]	0	3	5	6	9	10	14	16

Sum(2,5) = sum of items upto idx 5 – *// stored in index 6, i.e., index $(R+1)$*

sum of items upto idx 1 *// stored in index 2, i.e., index (L)*

Sum(L,R) = pref[R+1] – pref[L]

Query: find sum(L,R)

<u>Brute Force:</u>

```
for each Q query [L,R]:
      sum=0
      for i = L … R:
            sum+= a[i]
      print(sum)
```

Complexity: $O(?)$

Query: find sum(L,R)

<u>Brute Force:</u>

```
for each Q query [L,R]:
      sum=0
      for i = L … R:
            sum+= a[i]
      print(sum)
```

Complexity: $O(Q.N)$

Query: find sum(L,R)

<u>Brute Force:</u>

```
for each Q query [L,R]:
      sum=0
      for i = L … R:
            sum+= a[i]
      print(sum)
```

Complexity: $O(Q.N)$

Query: find sum(L,R)

<u>Prefix sum:</u>

```
for i = 1…N      // preprocessing
      pref[i] = pref[i-1] + a[i-1]

for every query [L,R]:
      print (pref[R+1] - pref[L])
```

Complexity: $O(?)$

Query: find sum(L,R)

<u>Brute Force:</u>

```
for each Q query [L,R]:
      sum=0
      for i = L … R:
            sum+= a[i]
      print(sum)
```

Complexity: $O(Q.N)$

Query: find sum(L,R)

<u>Prefix sum:</u>

```
for i = 1…N      // preprocessing
      pref[i] = pref[i-1] + a[i-1]

for every query [L,R]:
      print (pref[R+1] - pref[L])
```

Complexity: $O(N + Q)$

Given a string S of length N

Answer Q queries: given L and R, find the most frequent character in the substring $S[L, R]$

$$S = a\ b\ c\ c\ c\ a\ b\ a\ a\ a\ b\ b$$

Given a string S of length N

Answer Q queries: given L and R, find the most frequent character in the substring $S[L, R]$

$$S = a\ b\ c\ \boxed{c\ c\ a\ b\ a\ a}\ a\ b\ b$$

```
a occurs 3 times
b - 2
c – 2

Ans: ‘a’
```

Given a string S of length N

Answer Q queries: given L and R, find the most frequent character in the substring $S[L, R]$

$$S = a\ b\ c\ \boxed{c\ c\ a\ b\ a\ a}\ a\ b\ b$$

Can we apply Prefix sum concept to improve the situation?

Given a string S of length N

Answer Q queries: given L and R, find the most frequent character in the substring $S[L, R]$

$$S = a\ b\ c\ \boxed{c\ c\ a\ b\ a\ a}\ a\ b\ b$$

Can we apply Prefix sum concept to improve the situation?

Store freq of each letter upto each idx

Simply apply prefix sum formula!

Given a string S of length N

Answer Q queries: given L and R, find the most frequent character in the substring $S[L, R]$

$$S = a\ b\ c\ \boxed{c\ c\ a\ b\ a\ a}\ a\ b\ b$$

```
Useful idea while
working with lots of
Queries.
Just precompute
once!
```

Can we apply Prefix sum concept to improve the situation?

Can store freq of each letter upto each idx

Simply apply prefix sum formula!

Prefix sum doesn't work with min/max

idx	0	1	2	3	4	5	6	7
a[i]	20	7	6	7	4	30	2	
pref[i]	0	20	20	20	20	20	30	30

Find max of Range (2,4)

- When to use prefix sum?
  - Range sum
  - XOR, products
  - Frequency counts
  - Doesn’t work with min/max
  - Doesn’t support update operation

Sparse Table

Range Minimum Queries in $O(1)$

Sparse Table

- Efficiently answers range queries in an **immutable** array.
- Answer queries in $O(1)$ after $O(N\log N)$ preprocessing step.
- Stores ***range having length of power of 2*** instead of every possible range (hence the term 'Sparse')

Sparse Table

- **Best Use Case**: When the array is static (i.e., it doesn't change after initialization).

- **Ideal for**: Range Minimum Query, Range Maximum Query, and GCD/LCA-related range queries.

- **Not suitable for**: Problems that require frequent updates (use Segment Tree instead).

Key Features

- Preprocessing Time: $O(\mathrm{N}\log N)$
- Query Time: $O(1)$ for idempotent operations [1] (min, max, gcd, etc.)
- Space Complexity: $O(\mathrm{N}\log N)$

(1) Idempotent operations are actions in mathematics and computer science that produce the same result regardless of whether they are executed once or multiple times.

Concept

- A Sparse Table stores precomputed answers for overlapping ***ranges of powers of two***.

- For an array of size N, we define $table[i][j]$
  - i represents starting index of a range.
  - j represents <u>length of range as a power of 2</u> (i.e. 2^j) .

Concept

- A Sparse Table stores precomputed answers for overlapping ***ranges of powers of two***.

- For an array of size N, we define $table[i][j]$
  - i represents starting index of a range.
  - j represents <u>length of range as a power of 2</u> (i.e. 2^j)
  - $table[i][j]$ represents range $(i, i + 2^j - 1)$

Concept

- A Sparse Table stores precomputed answers for overlapping ***ranges of powers of two***.

- For an array of size N, we define $table[i][j]$
  - i represents starting index of a range.
  - j represents <u>length of range as a power of 2</u> (i.e. 2^j)
  - $table[i][j]$ represents range $(i, i + 2^j - 1)$

  - $table[3][2]$ represents range $(3, 3 + 2^2 - 1) = (3,6)$

Stores intervals of width $w = 2^j$

(1,1)
(1,2)
(1,3)
(1,4)
(1,5)
(1,6)
(1,7)
(1,8)

Will sparse table store precomputed results for all these ranges? (#items = 8)

Stores intervals of width $w = 2^j$

(1,1)
(1,2)
~~(1,3)~~
(1,4)
~~(1,5)~~
~~(1,6)~~
~~(1,7)~~
(1,8)

Will sparse table store precomputed results for all these ranges? (#items = 8)

No!
Only for the ones with range length corresponding power of two

Stores intervals of width $w = 2^j$

(1,1)	(2,2)
(1,2)	(2,3)
~~(1,3)~~	(2,4)
(1,4)	(2,5)
~~(1,5)~~	(2,6)
~~(1,6)~~	(2,7)
~~(1,7)~~	(2,8)
(1,8)	

Which $(2, x)$ ranges will be stored in Sparse Table?

Stores intervals of width $w = 2^j$

(1,1)	(2,2)
(1,2)	(2,3)
~~(1,3)~~	~~(2,4)~~
(1,4)	(2,5)
~~(1,5)~~	~~(2,6)~~
~~(1,6)~~	~~(2,7)~~
~~(1,7)~~	~~(2,8)~~
(1,8)	

Which $(2, x)$ ranges will be stored in Sparse Table?

Stores intervals of width $w = 2^j$

(1,1)	(2,2)
(1,2)	(2,3)
~~(1,3)~~	~~(2,4)~~
(1,4)	(2,5)
~~(1,5)~~	~~(2,6)~~
~~(1,6)~~	~~(2,7)~~
~~(1,7)~~	~~(2,8)~~
(1,8)	

How is this represented in sparse table?

table[i][j]=(i, i+2^j-1)

Stores intervals of width $w = 2^j$

(1,1)
(1,2)
~~(1,3)~~
(1,4)
~~(1,5)~~
~~(1,6)~~
~~(1,7)~~
(1,8)

(2,2)
(2,3)
~~(2,4)~~
(2,5)
~~(2,6)~~
~~(2,7)~~
~~(2,8)~~

How is this represented in sparse table?

table[i][j]=(i, i+2^j-1)

So,

table[2][0] represents range [2,2]

table[2][1] represents range [2,3]

table[2][2] represents range [2,5]

table[2][3] represents range [2,9]

...

This expands upto limit N (#items)

Which index of Sparse table corresponds to range (4,11)?

Which index of Sparse table corresponds to range (4,11)?

$$table[4][3] = (4, 4 + 2^3 - 1) = (4,11)$$

Which index of Sparse table corresponds to range (4,17)?

Which index of Sparse table corresponds to range (4,17)?

None!

windowsize = 17-4+1 = 14, which doesn’t correspond to power of two!

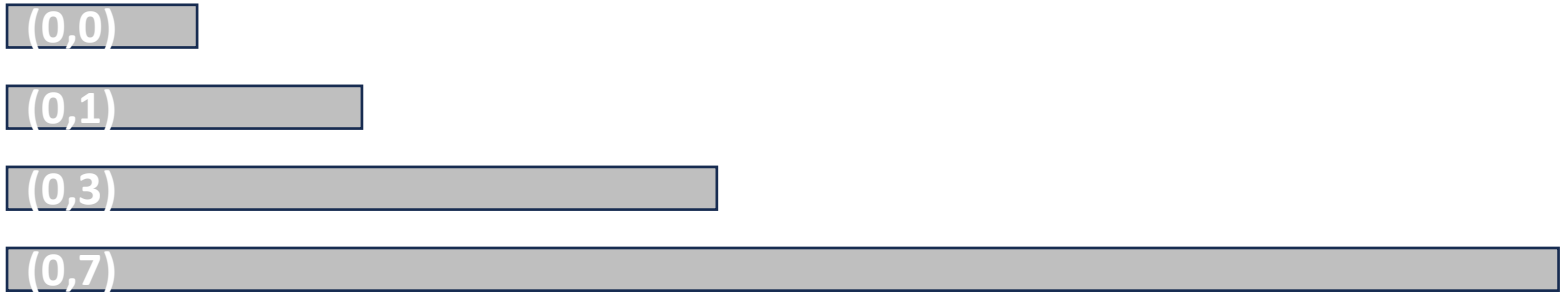
Which index of Sparse table corresponds to range (4,17)?

None!

windowsize = 17-4+1 = 14, which doesn't correspond to power of two!

// yet we can utilize the existing entries of the sparse table to efficiently compute this range…

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3



idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

(1,1)

(1,2)

(1,4)

(1,8) out of bound

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

(2,2)

(2,3)

(2,5)

(2,9) out of bound

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

(3,3)

(3,4)

(3,6)

(3,10) out of bound

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0				
1				
2				
3				
4				
5				
6				
7				

An Empty Sparse table

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	?			
1	?			
2	?			
3	?			
4	?			
5	?			
6	?			
7	?			

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1) = (0,0)$			
1	?			
2	?			
3	?			
4	?			
5	?			
6	?			
7	?			

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1) = (0,0)$			
1	(1,1)			
2	(2,2)			
3	(3,3)			
4	(4,4)			
5	(5,5)			
6	(6,6)			
7	(7,7)			

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1) = (0,0)$	?		
1	(1,1)	?		
2	(2,2)	?		
3	(3,3)	?		
4	(4,4)	?		
5	(5,5)	?		
6	(6,6)	?		
7	(7,7)	?		

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1) = (0,0)$	$(0,0+2^1-1) = (0,1)$		
1	(1,1)	?		
2	(2,2)	?		
3	(3,3)	?		
4	(4,4)	?		
5	(5,5)	?		
6	(6,6)	?		
7	(7,7)	?		

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	$(0,0+2^1-1)$ $=(0,1)$		
1	(1,1)	(1,2)		
2	(2,2)	(2,3)		
3	(3,3)	(3,4)		
4	(4,4)	(4,5)		
5	(5,5)	(5,6)		
6	(6,6)	(6,7)		
7	(7,7)	*Out of bound* ~~(7,8~~		

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	$(0,0+2^1-1)$ $=(0,1)$	$(0,0+2^2-1)$ $=(0,3)$	
1	(1,1)	(1,2)	(1,4)	
2	(2,2)	(2,3)	(2,5)	
3	(3,3)	(3,4)	(3,6)	
4	(4,4)	(4,5)	(4,7)	
5	(5,5)	(5,6)	~~(5,8)~~	
6	(6,6)	(6,7)	~~(6,9)~~	
7	(7,7)	*Out of bound* ~~(7,8~~	~~(7,10)~~	

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	$(0,0+2^1-1)$ $=(0,1)$	$(0,0+2^2-1)$ $=(0,3)$	$(0,0+2^3-1)$ $=(0,7)$
1	(1,1)	(1,2)	(1,4)	~~(1,8)~~
2	(2,2)	(2,3)	(2,5)	~~(2,9)~~
3	(3,3)	(3,4)	(3,6)	~~(3,10)~~
4	(4,4)	(4,5)	(4,7)	~~(4,11)~~
5	(5,5)	(5,6)	~~(5,8)~~	~~(5,12)~~
6	(6,6)	(6,7)	~~(6,9)~~	~~(6,13)~~
7	(7,7)	*Out of bound*~~(7,8~~	~~(7,10)~~	~~(7,14)~~

Which range does each position represent?

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	$(0,0+2^0-1)$ $=(0,0)$	$(0,0+2^1-1)$ $=(0,1)$	$(0,0+2^2-1)$ $=(0,3)$	$(0,0+2^3-1)$ $=(0,7)$
1	(1,1)	(1,2)	(1,4)	
2	(2,2)	(2,3)	(2,5)	
3	(3,3)	(3,4)	(3,6)	
4	(4,4)	(4,5)	(4,7)	
5	(5,5)	(5,6)		
6	(6,6)	(6,7)		
7	(7,7)			

Sparse table will save info of these ranges only

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	(0,1)	(0,3)	(0,7)
1	3	(1,2)	(1,4)	
2	4	(2,3)	(2,5)	
3	5	(3,4)	(3,6)	
4	3	(4,5)	(4,7)	
5	1	(5,6)		
6	1	(6,7)		
7	3			

Lets fill the exact values

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	(0,3)	(0,7)
1	3	3	(1,4)	
2	4	4	(2,5)	
3	5	3	(3,6)	
4	3	1	(4,7)	
5	1	1		
6	1	1		
7	3			

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	(0,3)	(0,7)
1	3	3	(1,4)	
2	4	4	(2,5)	
3	5	3	(3,6)	
4	3	1	(4,7)	
5	1	1		
6	1	1		
7	3			

table[3][1] represents $\min(3, 3 + 2^1 - 1)$
$= (3,4)$

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	1
1	3	3	3	
2	4	4	1	
3	5	3	1	
4	3	1	1	
5	1	1		
6	1	1		
7	3			

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	1
1	3	3	3	
2	4	4	1	
3	5	3	1	
4	3	1	1	
5	1	1		
6	1	1		
7	3			

table[2][2] represents
$\min(2, 2 + 2^2 - 1)$
$= (2,5)$

idx	0	1	2	3	4	5	6	7
$a[i]$	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	1
1	3	3	3	
2	4	4	1	
3	5	3	1	
4	3	1	1	
5	1	1		
6	1	1		
7	3			

How to build this table efficiently?

Can't just check minimum of every range using loop!

Slow!

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find $\min(5,26)$

Query width $= \ 26 - 5 + 1 \ = \ 22$

This information is not directly stored in the table

How to use precomputed ranges to derive this?

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find $\min(5,26)$

Query width $= 26 - 5 + 1 = 22$

This information is not directly stored in the table

How to use precomputed ranges to derive this?

Ranges stored in sparse table starting from 5:

$T[5][0]$ represents $(5,5)$
$T[5][1]$ represents $(5,6)$
$T[5][2]$ represents $(5,8)$
$T[5][3]$ represents $(5,12)$
$T[5][4]$ represents $(5,20)$

$T[5][5]$ represents $(5,36)$
// this range is out of bound as N=29 in this case, hence not stored in table.

We must utilize this precomputed information!

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find $\min(5,26)$

Query width $= 26 - 5 + 1 = 22$

This information is not directly stored in the table

How to use precomputed ranges to derive this?

Biggest precomputed range starting from 5 is $table[5][4] = [5,20]$

Ranges stored in sparse table starting from 5:

$T[5][0]$ represents $(5,5)$
$T[5][1]$ represents $(5,6)$
$T[5][2]$ represents $(5,8)$
$T[5][3]$ represents $(5,12)$
$T[5][4]$ represents $(5,20)$

$T[5][5]$ represents $(5,36)$
// this range is out of bound as N=29 in this case, hence not stored in table.

Now, what is the biggest precomputed range we can use?

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find $\min(5,26)$

Query width $= \ 26 - 5 + 1 \ = \ 22$

This information is not directly stored in the table

How to use precomputed ranges to derive this?

Now, what is the biggest precomputed range we can use:
$table[21][2]$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find $\min(5,26)$

Query width $= \ 26 - 5 + 1 \ = \ 22$
This information is not directly stored in the table

How to use precomputed ranges to derive this?

Finally use $table[25][1]$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find $\min(5,26)$

Query width $= \ 26 - 5 + 1 \ = \ 22$

This information is not directly stored in the table

How to use precomputed ranges to derive this?

Finally use $table[25][1]$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

$$\min(5,26) \;=\; \min(\, table[5][4], [21][2], [25][1])$$

This is possible for any range of width w, as

every number can be represented as a summations of power of 2.

e.g. here $w = 26 - 5 + 1 = 22 = (10110)_2$

Observe: $(10110)_2$ and how we used a range of 16, 4, 2 to find the total range

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

$$\min(5,26) \;=\; \min(\, table[5][4], [21][2], [25][1])$$

This is possible for any range of width w, as

every number can be represented as a summations of power of 2.

e.g. here $w = 26 - 5 + 1 = 22 = (10110)_2$

Observe: $(10110)_2$ and how we used a range of $16, 4, 2$ to find the total range

Complexity of answering range minimum query = $O(??)$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

$$\min(5,26) \;=\; \min(\, table[5][4], [21][2], [25][1])$$

This is possible for any range of width w, as

every number can be represented as a summations of power of 2.

e.g. here $w = 26 - 5 + 1 = 22 = (10110)_2$

Observe: $(10110)_2$ and how we used a range of 16, 4, 2 to find the total range

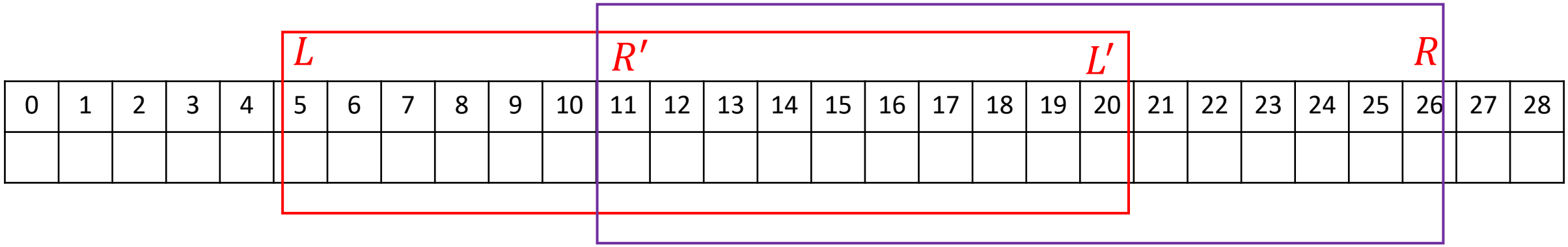
Complexity of answering range minimum query = $\boldsymbol{O(\log(w))}$

Can we do it in $\boldsymbol{O(1)}$***?***

					L															L'						R		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28

Find $\min(L, R) = \min(5,26)$
Window size=22

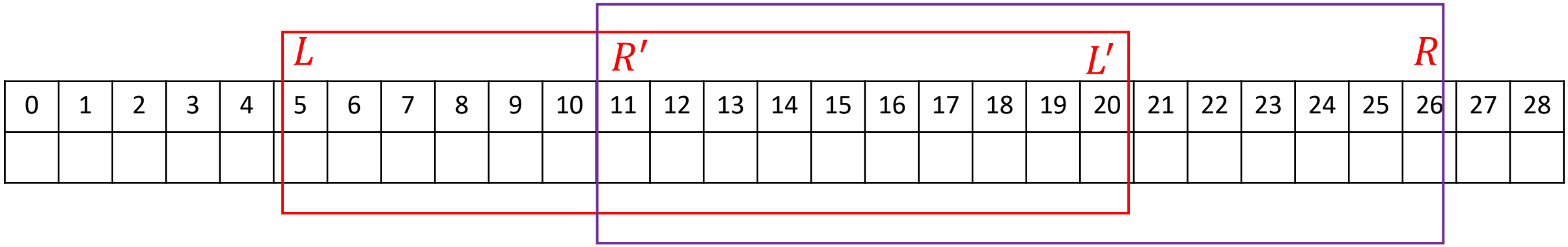
Biggest power of 2 less than 22 is 16, range $(5 + 2^4 - 1) = (5,20)$, representing table[5][4]



Find $\min(L, R) = \min(5,26)$
Window size=22

Biggest power of 2 less than 22 is 16, range $(5 + 2^4 - 1) = (5,20)$, representing table[5][4]

What if we use a similar size window starting from 16 blocks left to R? $(range\ (11,26) \sim table[11][4])$

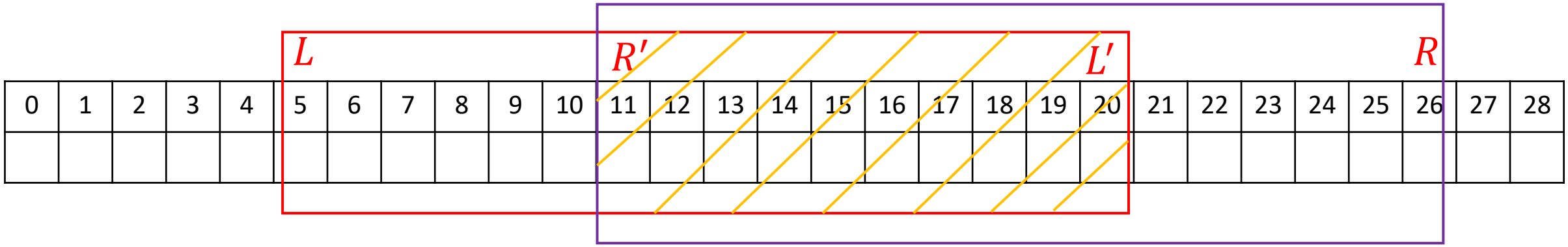


Find $\min(L, R) = \min(5,26)$
Window size=22

Biggest power of 2 less than 22 is 16, range $(5 + 2^4 - 1) = (5,20)$, representing table[5][4]

What if we use a similar size window starting from 16 blocks left to R? $(range\ (11,26) \sim table[11][4])$

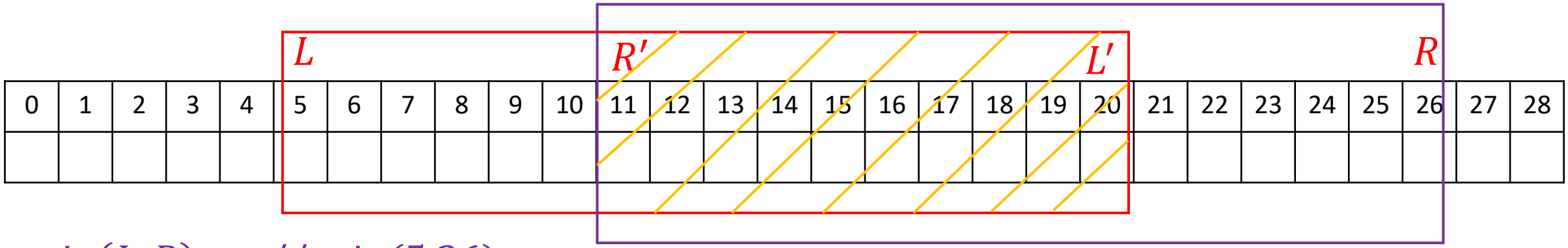
Now, $\min(5,26) = \min(\min(5,20), \min(11,26))$



Now, $\min(5,26) = \min(\min(5,20), \min(11,26))$

The overlapped region doesn't cause any harm as it doesn't change the minimum value!

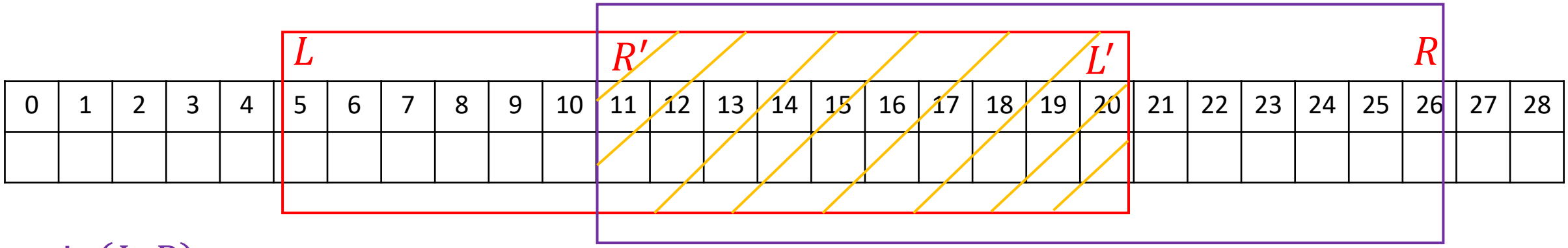
Time complexity for range minimum query is now $\boldsymbol{O}(\mathbf{1})$



$\min(L, R)$ // $\min(5,26)$

$= \min(\min(5,20), \min(11,26))$

$= \min(\min(L, L'), \min(R', R))$, *here* $L = 5, R = 26, L' = 20, R' = 11$

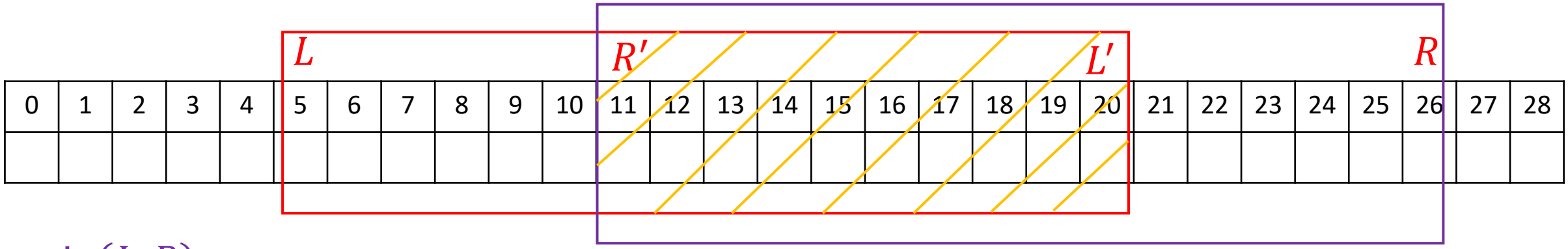


$\min(L, R)$
$= \min(\min(L, L'), \min(R', R)), here\ L = 5, R = 26, L' = 20, R' = 11$

How to find L' and R'?

It depends on $windowsize$

Here, $windowsize = R - L + 1 = 26 - 5 + 1 = 22$



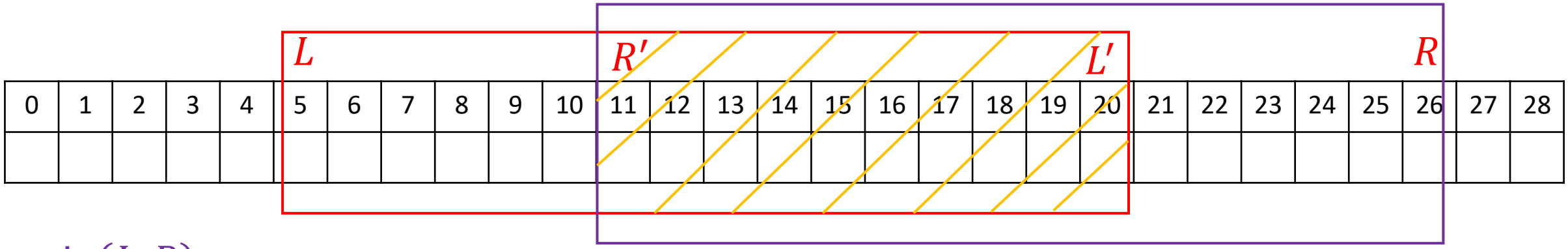
$\min(L, R)$
$= \min(\min(L, L'), \min(R', R)), here\ L = 5, R = 26, L' = 20, R' = 11$

How to find L' and R'?

It depends on $windowsize$

Here, $windowsize = R - L + 1 = 26 - 5 + 1 = 22$

Let, $j = \log(windowsize) = \log(22) = 4.459 \sim 4\ (floor)$



$\min(L, R)$
$= \min(\min(L, L'), \min(R', R)), here\ L = 5, R = 26, L' = 20, R' = 11$

How to find L' and R'?

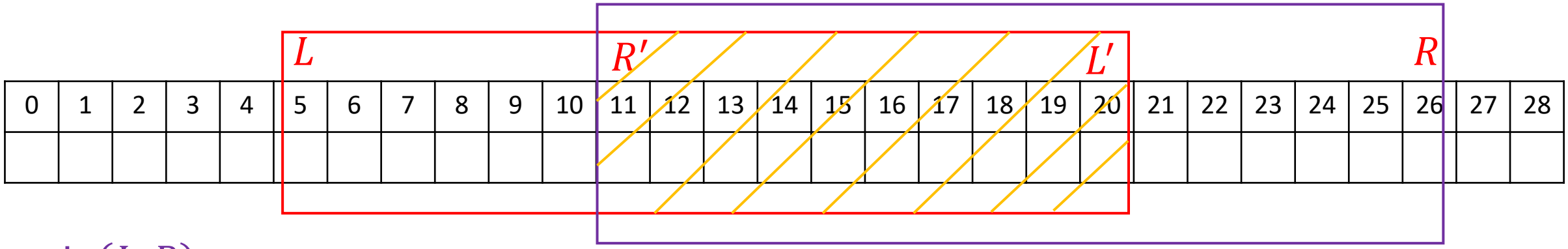
It depends on $windowsize$

Here, $windowsize = R - L + 1 = 26 - 5 + 1 = 22$

Let, $j = \log(windowsize) = \log(22) = 4.459 \sim 4\ (floor)$

$L' = 5 + 2^j - 1 = 20$

$Range$ (5,20) is represented by $table[5][4] \sim table[L][j]$

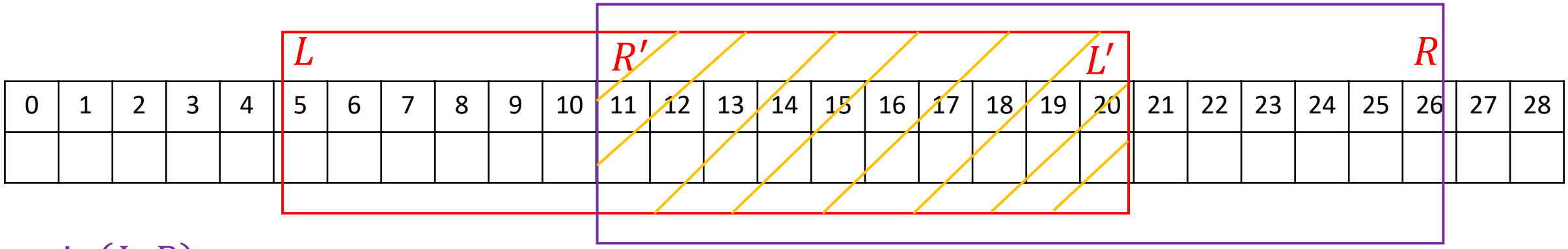


$\min(L, R)$
$= \min(\min(L, L'), \min(R', R)), here\ L = 5, R = 26, L' = 20, R' = 11$
$= \min(table[L][j], ?)$

Now, $R'(11)$ is 2^j steps to the left from $R(26)$

$$R' = R - 2^j + 1 = 26 - 2^4 + 1 = 11$$

$Range\ (R', R) \sim (11,26)$ is stored in $table[11][4] \sim table[R - 2^j + 1][j]$

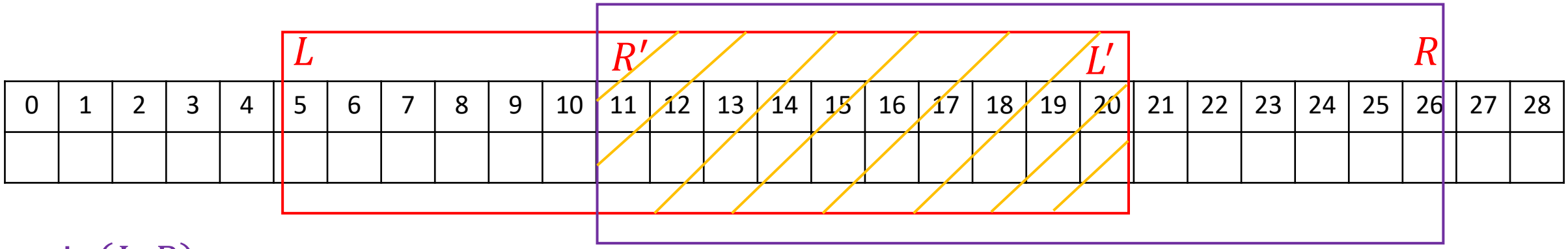


$$\min(L, R)$$
$$= \min(\min(L, L'), \min(R', R)), here\ L = 5, R = 26, L' = 20, R' = 11$$
$$= \min(table[L][j], table[R - 2^j + 1][j])$$

Now, $R'(11)$ is 2^j steps to the left from $R(26)$

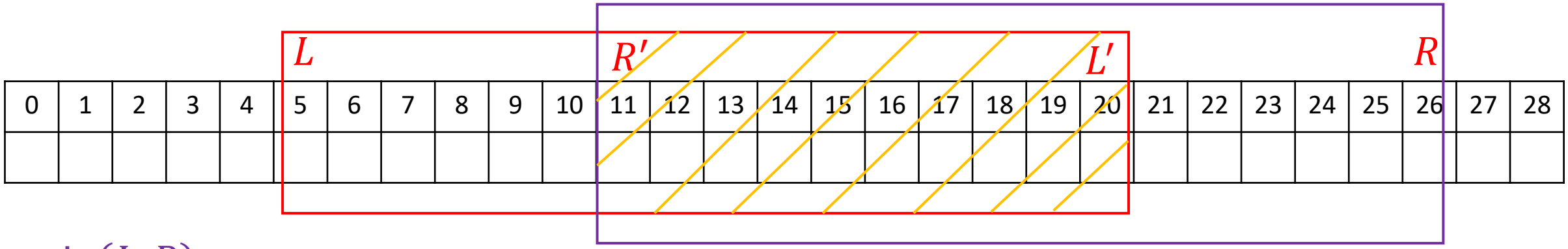
$$R' = R - 2^j + 1 = 26 - 2^4 + 1 = 11$$

$Range\ (R', R) \sim (11,26)$ is stored in $table[11][4] \sim table[R - 2^j + 1][j]$



$$\min(L, R)$$
$$= \min(\min(L, L'), \min(R', R)), here\ L = 5, R = 26, L' = 20, R' = 11$$
$$= \min(table[L][j], table[R - 2^j + 1][j])$$
$$= \min(table[L][j], table[R - 1 \ll j + 1][j])$$

2^j is equivalent to $1 \ll j$ (bitwise left-shifting)



$$\min(L, R)$$
$$= \min(\min(L, L'), \min(R', R)), here\ L = 5, R = 26, L' = 20, R' = 11$$
$$= \min(table[L][j], table[R - 2^j + 1][j])$$
$$= \min(table[L][j], table[R - 1 \ll j + 1][j])$$

Reminder:

- *The overlapped region doesn't change the minimum value!*
- *Time complexity for range minimum query is now* $\boldsymbol{O(1)}$

```
int main() {
        // 1) read input
        int n; cin >> n;
        for(int i = 0; i < n; i++) {
                cin >> a[i];
                table[i][0] = a[i];
        }

}
```

```
int main() {
        // 1) read input
        int n; cin >> n;
        for(int i = 0; i < n; i++) {
                cin >> a[i];
                table[i][0] = a[i];
        }
        // 2) preprocessing, O(N*log(N))
        for(int j = 1; j < log(n); k++) {
                for(int i = 0; i + (1 << j) - 1 < n; i++) {
                        table[i][j] = min(table[i][j-1],
                                          table[i+(1<<(j-1))][j-1]);
                }
        }
}
```

```
int main() {
        // 1) read input
        int n; cin >> n;
        for(int i = 0; i < n; i++) {
                cin >> a[i];
                table[i][0] = a[i];
        }
        // 2) preprocessing, O(N*log(N))
        for(int j = 1; j < log(n); k++) {
                for(int i = 0; i + (1 << j) - 1 < n; i++) {
                        table[i][j] = min(table[i][j-1],
                                          table[i+(1<<(j-1))][j-1]);
                }
        }
        // 3) answer queries
        int q; cin >> q;
        for(int i = 0; i < q; i++) {
                int L, R; cin >> L >> R;
                cout << query(L, R) << "\n";
        }
}
```

```
int main() {
        // 1) read input
        int n; cin >> n;
        for(int i = 0; i < n; i++) {
                cin >> a[i];
                table[i][0] = a[i];
        }
        // 2) preprocessing, O(N*log(N))
        for(int j = 1; j < log(n); k++) {
                for(int i = 0; i + (1 << j) - 1 < n; i++) {
                        table[i][j] = min(table[i][j-1],
                                          table[i+(1<<(j-1))][j-1]);
                }
        }
        // 3) answer queries
        int q; cin >> q;
        for(int i = 0; i < q; i++) {
                int L, R; cin >> L >> R;
                cout << query(L, R) << "\n";
        }
}
```

```
int query(int L, int R) { // O(1)
        int window = R - L + 1;
        int j = log(window);
        return min(
                table[L][j],
                table[R-(1<<j)+1][j]);
}
```

idx	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2			
1	3			
2	4			
3	5			
4	3			
5	1			
6	1			
7	3			

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2			
1	3			
2	4			
3	5			
4	3			
5	1			
6	1			
7	3			

1) read input

```
int n; cin >> n;
for(int i = 0; i < n; i++){
  cin >> a[i];
  table[i][0] = a[i];
}
```

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2			
1	3			
2	4			
3	5			
4	3			
5	1			
6	1			
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3			
2	4			
3	5			
4	3			
5	1			
6	1			
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 0$

$table[0][1]$ (represents $range$ $(0,0 + 2^1 - 1) = (0,1)$

$= \min(table[0][0], table[0 + 1 << 0][1 - 1])$

$= \min(table[0][0], table[1][0])$

$= \min(2,3)$

$= 2$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4			
3	5			
4	3			
5	1			
6	1			
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 0$

$table[0][1]$(represents $range$ $(1, 1 + 2^1 - 1) = (1,2)$

$= \min(table[1][0], table[1 + 1 << 0][1 - 1])$

$= \min(table[1][0], table[2][0])$

$= \min(3,4)$

$= 3$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5			
4	3			
5	1			
6	1			
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 2$

$table[2][1]$ (represents $range$ $(2,2+2^1-1) = (2,3)$

$= \min(table[2][0], table[2+1 << 0][1-1])$

$= \min(table[2][0], table[3][0])$

$= \min(4,5)$

$= 4$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5	3		
4	3			
5	1			
6	1			
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 3$

$table[3][1]$ (represents $range$ $(3,3 + 2^1 - 1) = (3,4)$

$= \min(table[3][0], table[3 + 1 << 0][1 - 1])$

$= \min(table[3][0], table[4][0])$

$= \min(5,3)$

$= 3$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1			
6	1			
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 4$

$table[4][1]$ (represents $range$ $(4,4 + 2^1 - 1) = (4,5)$

$= \min(table[4][0], table[4 + 1 << 0][1 - 1])$

$= \min(table[4][0], table[5][0])$

$= \min(3,1)$

$= 1$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1			
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 5$

$table[5][1]$ (represents $range$ $(5,5 + 2^1 - 1) = (5,6)$

$= \min(table[5][0], table[5 + 1 << 0][1 - 1])$

$= \min(table[5][0], table[6][0])$

$= \min(1,1)$

$= 1$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1	1		
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 6$

$table[6][1]$ (represents $range\ (6, 6 + 2^1 - 1) = (6,7)$

$= \min(table[6][0], table[6 + 1 << 0][1 - 1])$

$= \min(table[6][0], table[7][0])$

$= \min(1,3)$

$= 1$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2		
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1	1		
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 1, i = 7$

As $i + (1 \ll j) = 7 + 1 \ll 1 = 7 + 2 = 9 > n$

Inner loop breaks

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1	1		
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

$j = 2, i = 0$

$table[0][2]$ (represents $range\ (0, 0 + 2^2 - 1) = (0,3)$

$= \min(table[0][2-1], table[0 + 1 \ll 1][2-1])$

$= \min(table[0][1], table[2][1])$

$= \min(range(0,1), range(2,3))$

$= \min(2,4)$

$= 2$

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1	1		
7	3			

Using the precomputed info, now we can efficiently find the min in O(1) even during build procedure!

idx	0	1	2	3	4	5	6	7
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	
1	3	3		
2	4	4		
3	5	3		
4	3	1		
5	1	1		
6	1	1		
7	3			

2) preprocessing, O(N*log(N))

```
for(int j=1; j<=log(n); k++) {
  for(int i=0; i+ (1<<j) <= n; i++){
    table[i][j] =
        min(table[i][j-1],
            table[i+(1<<(j-1))][j-1]);
  }
}
```

Try to complete the remaining table by yourself…

idx	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
a[i]	2	3	4	5	3	1	1	3

i	j=0 (2^0)	j=1 (2^1)	j=2 (2^2)	j=3 (2^3)
0	2	2	2	1
1	3	3	3	
2	4	4	1	
3	5	3	1	
4	3	1	1	
5	1	1		
6	1	1		
7	3			

Thank you